

RTSUG,
July 16, 2026

We Are Family – Practical Tips for Integrating Python, R, and SAS Under One Roof

Chevell Parker,
SAS Institute



Overview

- Integrating Python, R, and SAS
- Analyzing Data in the Cloud using Python and R
- Introducing the SAS Viya Workbench

Integrating Python, R and SAS with Viya 4



- SWAT
- SASPY
- PROC PYTHON
- PROC GATEWAY
- PROC FCMP
- SAS Studio Flows
- SAS Studio Python Editor
- Model Studio OSCN
- SAS Configurator for Open Source
- SAS Event Stream Processing
- Micro Analytic Service



- R SWAT
- PROC R
- PROC IML
- PROC GATEWAY
- Model Studio OSCN
- SAS Configurator for Open Source
- SAS Studio R Editor



- Cloud Analytics Services
- Compute Server



Integrating Python, R, and SAS

- Integrating Python and R with your Viya 4 Applications
- Using the SAS Language for Python and R Integration
- Managing Your Python and R Environments
- Troubleshooting Common Integration Issues

Integrating Python and R with your Viya 4 Applications

SAS Studio Python Editor

SAS® Studio - Develop Code and Flows

New Options View Open Save All

Start Page R.r +

Run Cancel Up Down Undo Redo Clear Copy to flow Snippet

Code Log Results Output Data

```
1 require(graphics)
2 pairs(mtcars, main = "mtcars data", gap = 1/4)
3 coplot~mpg ~ disp | as.factor(cyl), data = mtcars,
4       panel = panel.smooth, rows = 1)
5 ## possibly more meaningful, e.g., for summary() or bivariate
6 mtcars2 <- within(mtcars, {
7   vs <- factor(vs, labels = c("V", "S"))
8   am <- factor(am, labels = c("automatic", "manual"))
9   cyl <- ordered(cyl)
10  gear <- ordered(gear)
11  carb <- ordered(carb)
12 })
13 summary(mtcars2)
14
15
16
```

⊗ (0) ⚠ (0) ⓘ (10)

There are no messages.

mpg	cyl	disp	hp	drat
Min. :10.40	4:11	Min. : 71.1	Min. : 52.0	Min. :2.76
1st Qu.:15.43	6: 7	1st Qu.:120.8	1st Qu.: 96.5	1st Qu.:3.08
Median :19.20	8:14	Median :196.3	Median :123.0	Median :3.69
Mean :20.09		Mean :230.7	Mean :146.7	Mean :3.59
3rd Qu.:22.80		3rd Qu.:326.0	3rd Qu.:180.0	3rd Qu.:3.92
Max. :33.90		Max. :472.0	Max. :335.0	Max. :4.93

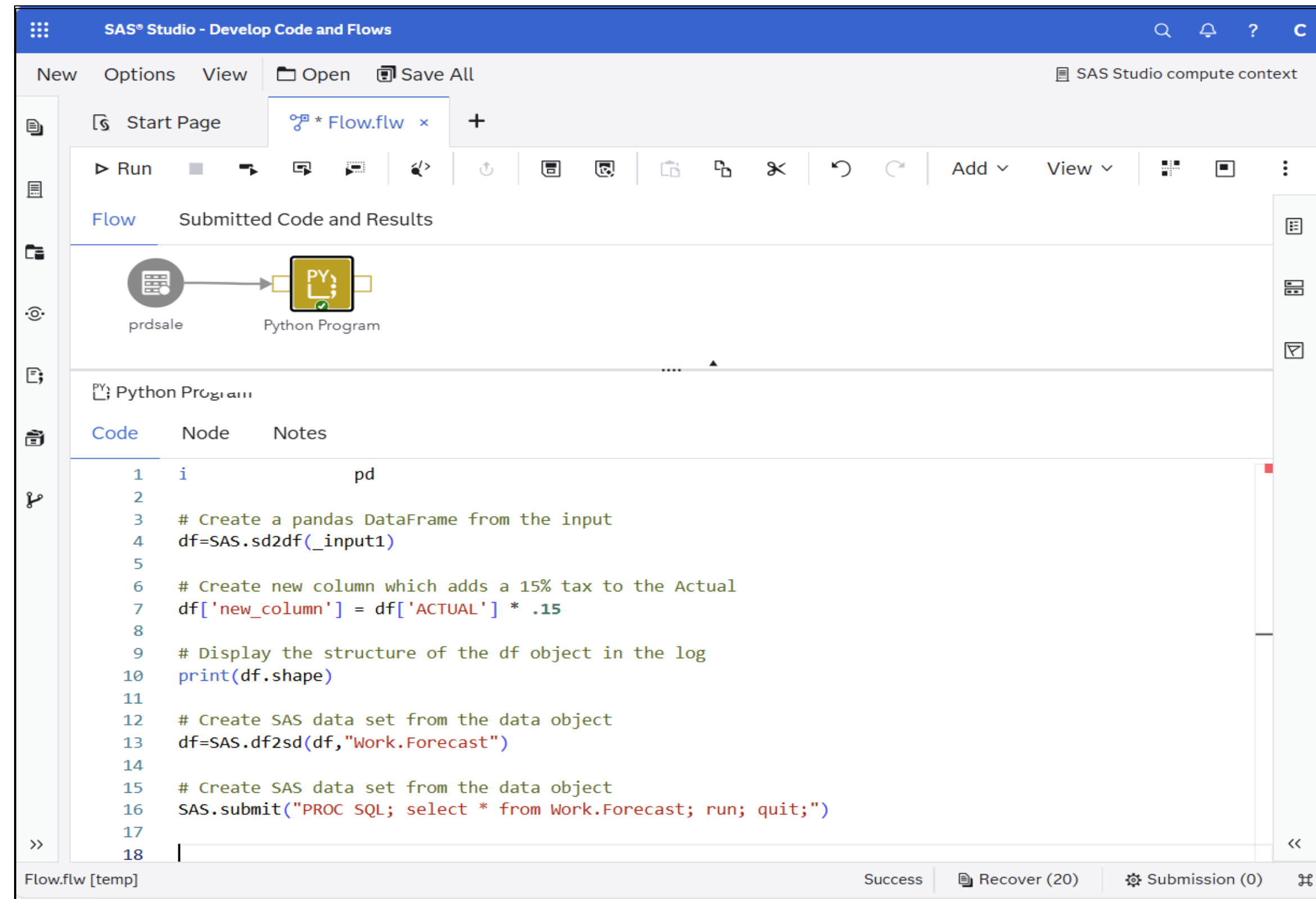
wt	qsec	vs	am	gear	carb
Min. :1.513	Min. :14.50	V:18	automatic:19	3:15	1: 7
1st Qu.:2.581	1st Qu.:16.89	S:14	manual :13	4:12	2:10
Median :3.325	Median :17.71			5: 5	3: 3
Mean :3.217	Mean :17.85				4:10
3rd Qu.:3.610	3rd Qu.:18.90				6: 1
Max. :5.424	Max. :22.90				8: 1

NOTE: PROCEDURE R used (Total process time):
real time 0.30 seconds
cpu time 0.11 seconds



Integrating Python and R with your Viya 4 Applications

SAS Studio Flows Program Step



The screenshot displays the SAS Studio interface for developing code and flows. The top menu bar includes 'New', 'Options', 'View', 'Open', and 'Save All'. The main workspace shows a flow diagram with a 'prdsale' node connected to a 'Python Program' node. Below the flow diagram, the 'Python Program' step is expanded, showing a list of code steps. The code is as follows:

```
1 i pd
2
3 # Create a pandas DataFrame from the input
4 df=SAS.sd2df(_input1)
5
6 # Create new column which adds a 15% tax to the Actual
7 df['new_column'] = df['ACTUAL'] * .15
8
9 # Display the structure of the df object in the log
10 print(df.shape)
11
12 # Create SAS data set from the data object
13 df=SAS.df2sd(df, "Work.Forecast")
14
15 # Create SAS data set from the data object
16 SAS.submit("PROC SQL; select * from Work.Forecast; run; quit;")
17
18
```

The bottom status bar shows 'Flow.flw [temp]' and 'Success Recover (20) Submission (0)'.

Integrating Python and R with your Viya 4 Applications

Model Studio OSCN

Summary

Output Data

Node

Assessment

R Code

```
1 #-----
2 # Language: R
3 #-----
4 # Set R-work to node-work directory
5 dm_nodedir <- '/opt/sas/viya/config/var/tmp/compsrv/default/e0fea'
6 setwd(dm_nodedir)
7
8 # Variable declarations
9 dm_dec_target <- 'BAD'
10 dm_partitionvar <- '_PartInd_'
11 dm_partition_train_val <- 1
12
13 dm_class_input <- c("IMP_DELINQ","IMP_DEROG","IMP_JOB","IMP_NINQ")
14 dm_interval_input <- c("IMP_CLAGE","IMP_CLNO","IMP_DEBTINC","IMP_I
15
```

R Output

```
1
2 Call:
3 glm(formula = BAD ~ IMP_CLAGE + IMP_CLNO + IMP_DEBTINC + IMP_LOAN
4     IMP_MORTDUE + IMP_VALUE + IMP_YOJ, family = binomial(link = "
5     data = dm_traindf)
6
7 Deviance Residuals:
8      Min       1Q   Median       3Q      Max
9 -1.1035  -0.7089  -0.5776  -0.4000   3.9216
10
11 Coefficients:
12              Estimate Std. Error z value Pr(>|z|)
13 (Intercept) -1.765e+00  2.637e-01  -6.694 2.17e-11 ***
14 IMP_CLAGE   -6.307e-03  6.544e-04  -9.637 < 2e-16 ***
15
```

Properties

Property Name	Property Value
osCode_Language	r
sampMethod	STRATIFY
sampType	NUM
sampObs	10,000
sampPercent	10
exportWithFormat	true

Output

Predicted Variables

Obs	Predicted Variable Name	Variable Name	Level Frequency	Level Frequency Percent	Raw Numeric Value	Raw Character Value	Formatted Value
1	P_BAD1	BAD	1189	19.9497	1		1
2	P_BAD0	BAD	4771	80.0503	0		0
3	I_BAD	BAD	.	.	.		



Integrating Python and R with your Viya 4 Applications

SAS Studio R Editor

SAS® Studio - Develop Code and Flows

NewOptionsViewOpenSave All

Start PageSAS Program.sasPython.pyPython 1.py

RunCancelCopy to flowSnippet

Code

LogResultsOutput Data (1)

```
1
2 import pandas as pd
3 from sklearn import datasets
4
5 # Load the Iris dataset from sklearn as an example
6 iris = datasets.load_iris()
7 df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
8
9 # Example species assignment
10 df['species'] = iris.target_names[iris.target[0]]
11
12 # Print the structure of the data object DF
13 print(df.shape)
14
15 # Convert the Pandas DataFrame to a SAS Data set
16 ds = SAS.df2sd(df, "WORK.IRIS_UPDATE")
17
18 # Print the first 10 observations from the SAS data set
19 SAS.submit("PROC PRINT DATA=IRIS_UPDATE(obs=10);RUN;")
20
21
```

Obs	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa
7	4.6	3.4	1.4	0.3	setosa
8	5.0	3.4	1.5	0.2	setosa
9	4.4	2.9	1.4	0.2	setosa
10	4.9	3.1	1.5	0.1	setosa



Using the SAS Language for Python and R Integration

- Python Integration
 - PROC PYTHON
 - PROC FCMP (Python Objects)
 - PROC GATEWAY
- R Integration
 - PROC R
 - PROC IML
 - PROC GATEWAY

The PYTHON Procedure

- Language Fundamentals
- Callback Methods and Interaction
- Integration Examples that Bridge the Divide

The Python Procedure Fundamentals

The PYTHON Procedure Overview

- Supported with Viya 4 and SAS 9.4M9
- Runs Python code directly within SAS programs
- Launches a sub-process from the Compute Server with Viya 4
- Leverages open-source libraries alongside SAS analytics
- Generates a seamless workflow between SAS and Python

The PYTHON Procedure 9.4M9

SAS Support - The PYTHON procedure is available for SAS® 9.4M9 (TS1M9)

Hot Fix Information

[Click here for the latest Hot Fix Information for KB0046084](#)

KB0046084

The PYTHON procedure is available for SAS® 9.4M9 (TS1M9)

Updated 📅 4d ago • 👁 249 Views • ★ ★ ★ ★ ★

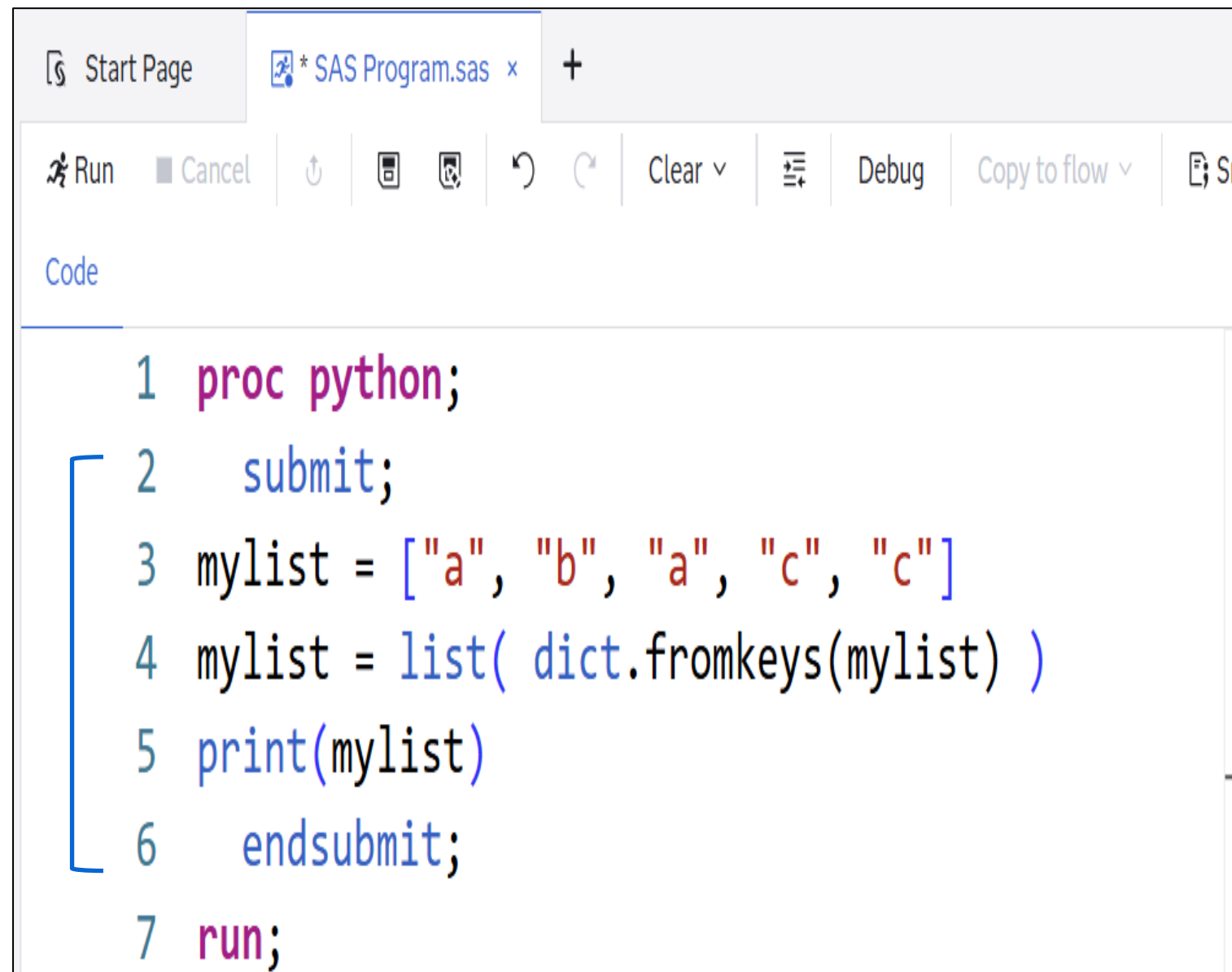
PROC PYTHON is available for SAS 9.4M9 on the following operating systems:

- Windows
- Windows for x64
- 64-bit Enabled AIX
- Linux for x64

For additional information, see [Configuring SAS 9.4 Foundation to Use PROC PYTHON](#).

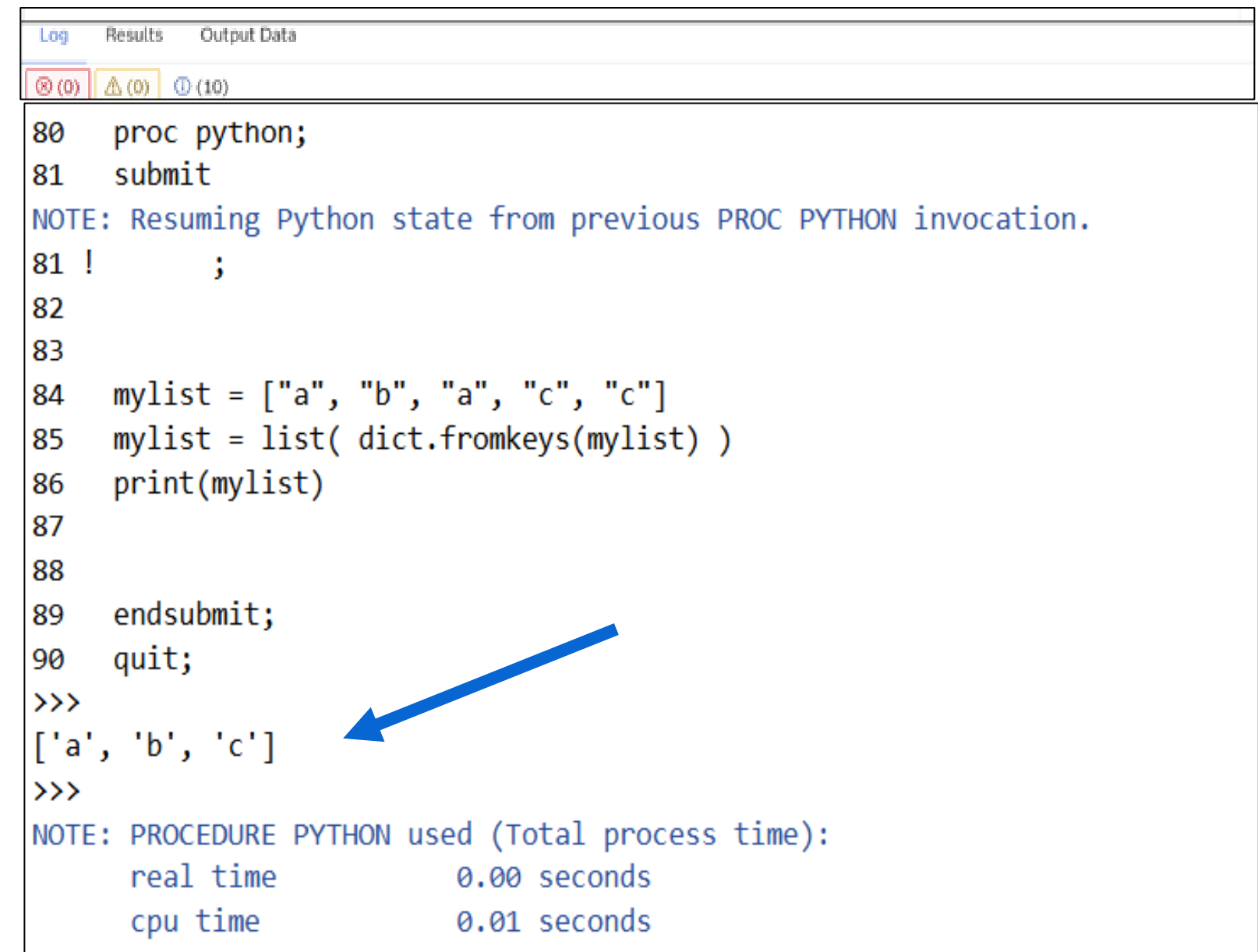
[Copy Permalink](#)

PROC PYTHON: Embedding Python Code



The screenshot shows the SAS IDE editor with a single tab titled '* SAS Program.sas'. The toolbar includes buttons for Run, Cancel, and various file and editing functions. The code editor contains the following SAS code:

```
1 proc python;  
2   submit;  
3   mylist = ["a", "b", "a", "c", "c"]  
4   mylist = list( dict.fromkeys(mylist) )  
5   print(mylist)  
6   endsubmit;  
7 run;
```



The screenshot shows the SAS IDE output window with tabs for Log, Results, and Output Data. The Log tab is active, displaying the execution of the PROC PYTHON code. The output includes the SAS code lines, a note about resuming the Python state, the Python code execution, and the resulting list output. A blue arrow points to the output list.

```
80 proc python;  
81 submit  
NOTE: Resuming Python state from previous PROC PYTHON invocation.  
81 !      ;  
82  
83  
84 mylist = ["a", "b", "a", "c", "c"]  
85 mylist = list( dict.fromkeys(mylist) )  
86 print(mylist)  
87  
88  
89 endsubmit;  
90 quit;  
>>>  
['a', 'b', 'c']  
>>>  
NOTE: PROCEDURE PYTHON used (Total process time):  
      real time          0.00 seconds  
      cpu time           0.01 seconds
```

PROC PYTHON: External File Execution

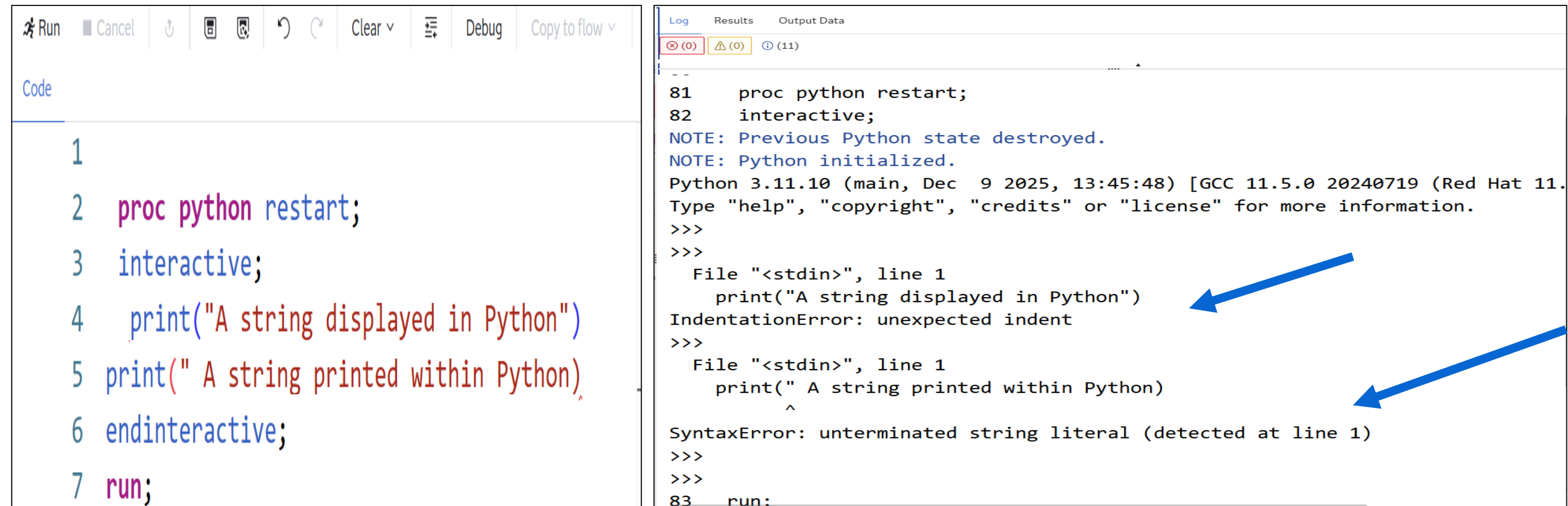
The screenshot displays the SAS Studio interface. On the left, the 'SAS Server' pane shows the file hierarchy under 'Home', with 'sample2.py' highlighted. The main editor window shows a SAS program with two lines of code:

```
1 proc python infile="/srv/nfs/compute/home/sasctp/sample2.py";  
2 run;
```

Below the code editor, the 'Log' pane shows the execution output:

```
>1 /* region: Generated preamble */ ...  
79  
80 proc python infile="/srv/nfs/compute/home/sasctp/sample2.py";  
81 run;  
NOTE: Resuming Python state from previous PROC PYTHON invocation.  
>>>  
['a', 'b', 'c']  
>>>  
NOTE: PROCEDURE PYTHON used (Total process time):  
      real time          0.01 seconds  
      cpu time           0.00 seconds
```


PROC PYTHON: Using the Interactive Statement



The screenshot displays the SAS Studio interface. On the left, the 'Code' editor contains the following SAS code:

```
1  
2 proc python restart;  
3 interactive;  
4 print("A string displayed in Python")  
5 print(" A string printed within Python")  
6 endinteractive;  
7 run;
```

On the right, the 'Log' window shows the execution output. It includes the following text:

```
--  
81 proc python restart;  
82 interactive;  
NOTE: Previous Python state destroyed.  
NOTE: Python initialized.  
Python 3.11.10 (main, Dec 9 2025, 13:45:48) [GCC 11.5.0 20240719 (Red Hat 11.  
Type "help", "copyright", "credits" or "license" for more information.  
>>>  
>>>  
File "<stdin>", line 1  
    print("A string displayed in Python")  
IndentationError: unexpected indent  
>>>  
File "<stdin>", line 1  
    print(" A string printed within Python")  
      ^  
SyntaxError: unterminated string literal (detected at line 1)  
>>>  
>>>  
83 run;
```

Two blue arrows point from the right side of the image to the error messages in the log: one points to the 'IndentationError' and the other points to the 'SyntaxError'.

The Python Procedure Callback Methods

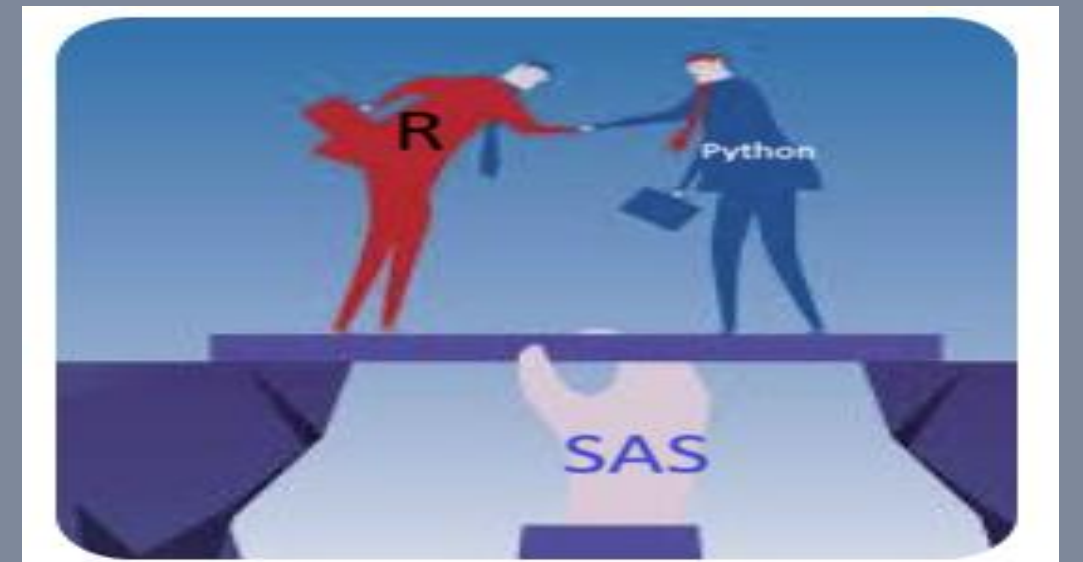


Callback Methods Overview

- Transfers data between a Pandas DataFrame and a SAS data set
- Creates and retrieves SAS macro variables from a Python session
- Invokes SAS functions or functions generated with PROC FCMP
- Executes SAS code from within the Python session
- Provides the ability to view the SAS Log during Execution

Callback Methods

Method	Results
<code>SAS.df2sd(dataframe,'libref.dataset')</code>	Transfers data from a Pandas DataFrame to a SAS data set
<code>SAS.sd2df(dataframe,'libref.dataset')</code>	Transfers a SAS data set to a Pandas DataFrame
<code>SAS.symput("macro-var",value)</code>	Assigns a Python value to SAS macro variable in the global scope
<code>SAS.symget("macro-var")</code>	Retrieves the value of a SAS macro variable within the Python environment
<code>SAS.submit("sas-code")</code>	Executes SAS code directly from Python
<code>SAS.sasfnc("function-name",args)</code>	Invokes SAS functions or functions generated with PROC FCMP
<code>SAS.hideLOG(True)</code>	Hide the SAS Log (PROC PRINTTO)



Bridging the Divide Using the Python Procedure

Leveraging SAS Procedures from Python

```
Run Cancel [Icons] Clear ▾ [Icons] Debug Copy to flow ▾ [Icon]
Code
1  proc python;
2  submit;
3  import pandas as pd
4
5  # Create data object from CSV file
6  df= pd.read_csv('https://data.wprdc.org/dataset/'
7                  '0b584c84-7e35-4f4d-a5a2-b01697470c0f/resource/'
8                  'f2439bd4-848f-46f7-bd19-a21853751dd3/download/avsurvey2019data.csv'
9  )
10 # Transfer DataFrame to a SAS data set
11 SAS.df2sd(df,"rideshare")
12
13 # Create Global macro variable
14 SAS.symput('myvar',"Report generated on &sysdate")
15
16 # Submit code to SAS
17 SAS.submit("proc freq data=rideshare;"
18            "table provingGround*age / out=rideshare_out;"
19            "title ""&myvar""; "
20            "run;"
21 )
22 SAS.sd2df("rideshare_out","rideshare_out")
23 endsubmit;
```

Leveraging SAS Procedures from Python (Continued)

Report generated 11MAR26

The FREQ Procedure

Frequency
Percent
Row Pct
Col Pct

Table of ProvingGround by Age								
ProvingGround	Age							Total
	18-24	25-34	35-44	45-54	55-64	65+	Under 18	
Approve	19	94	78	61	71	57	0	380
	2.42	11.97	9.94	7.77	9.04	7.26	0.00	48.41
	5.00	24.74	20.53	16.05	18.68	15.00	0.00	
	55.88	45.85	44.32	47.66	49.31	58.76	0.00	
Disapprove	3	15	13	15	10	9	0	65
	4.62	23.08	20.00	23.08	15.38	13.85	0.00	8.28
	8.82	7.32	7.39	11.72	6.94	9.28	0.00	
Neutral	5	18	18	14	27	9	0	91
	0.64	2.29	2.29	1.78	3.44	1.15	0.00	11.59
	5.49	19.78	19.78	15.38	29.67	9.89	0.00	
	14.71	8.78	10.23	10.94	18.75	9.28	0.00	
Somewhat Approve	5	47	43	30	26	15	1	167
	0.64	5.99	5.48	3.82	3.31	1.91	0.13	21.27
	2.99	28.14	25.75	17.96	15.57	8.98	0.60	
	14.71	22.93	24.43	23.44	18.06	15.46	100.00	
Somewhat Disapprove	2	31	24	8	10	7	0	82
	0.25	3.95	3.06	1.02	1.27	0.89	0.00	10.45
	2.44	37.80	29.27	9.76	12.20	8.54	0.00	
	5.88	15.12	13.64	6.25	6.94	7.22	0.00	
Total	34	205	176	128	144	97	1	785
	4.33	26.11	22.42	16.31	18.34	12.36	0.13	100.00
Frequency Missing = 10								

Transfer dataFrame to a SAS data set

Leveraging Python's ability to add Long Strings

```
import pandas as pd
import textwrap
```

```
# Generate sample data with long strings
```

```
data = {'TextColumn': [
    'This is a very long string that needs wrapping',
    'Short string',
    'Another very long string to test the wrapping capability'
]}
df = pd.DataFrame(data)
limit = 15
```

```
# Generate new columns based on text wrap
```

```
wrapped_series = df['TextColumn'].apply(lambda x: textwrap.wrap(x, width=limit))
max_lines = wrapped_series.apply(len).max()
for i in range(max_lines):
    df[f'Wrapped_{i+1}'] = wrapped_series.apply(lambda x: x[i] if i < len(x) else "")
```

```
# Transfer the Pandas DataFrame to a SAS data sets and print it
```

```
SAS.df2sd(df,"Long_chars(keep=Wrapped_1—Wrapped_4")
SAS.submit("PROC PRINT DATA=Long_chars; run;")
```


Leveraging Python's ability to add Long Strings (continued)

Log	Results	Output Data (1)		
Obs	Wrapped_1	Wrapped_2	Wrapped_3	Wrapped_4
1	This is a very long	string that needs	wrapping before moving to	SAS
2	This string is much	shorter		
3	This String does is	another string that will	need wrapping	

Leveraging Python's JSON Libraries

JSONL

```
{ "name": "John", "session": 2013, "score": 24, "Completed": 1}  
{ "name": "Alexa", "session": 2013, "score": 27, "Completed": 1}  
{ "name": "May", "session": 2012, "score": 14, "Completed": 0}  
{ "name": "John", "session": 2013, "score": 18, "Completed": 1}
```

Leveraging Python's JSON Libraries (continued)

JSON

Run

Cancel

Clear

Debug

Copy to f

Code

1

proc python;

2

submit;

3

4

import pandas as pd

5

6

Read JSONL file creating a data object

7

df=pd.read_ison('/srv/nfs/compute/home/sasctp/json_sample.jsonl',lines=True) ;

8

9

Using the callback method create a SAS dtaa set

10

SAS.df2sd(df,"Classdata")

11

12

Using the callback method print the table

13

SAS.submit("Proc print data=work.classdata;run;")

14

endsubmit;

15

run;

Log

Results

Output Data (1)

Obs	name	session	score	completed
1	John	2013	24	1
2	Alexa	2013	27	1
3	May	2012	14	0
4	John	2012	18	1

Leveraging Python's Libraries for Reading Parquet Files

The screenshot displays the SAS Studio interface. The top blue header bar contains the SAS logo, the text "SAS® Studio - Develop Code and Flows", and a search icon. Below the header is a menu bar with "New", "Options", "View", "Open", and "Save All". The left sidebar shows the "SAS Server" tree with a file explorer view. The main workspace has a tab for "SAS Program.sas" and a toolbar with icons for Run, Submit, and other actions. The code editor contains the following Python code:

```
1 proc python;
2 submit;
3 import pandas as pd
4 # Read parquet file
5 df = pd.read_parquet('/srv/nfs/compute/home/sasctp/titanic.parquet')
6 # Copy DataFrame to SAS data set
7 SAS.df2sd(df,"titanic")
8 endsubmit;
9 run;
10
11 proc print data=titanic(obs=5);
12 run;
```

Leveraging Python's Libraries for Reading Paquet Files (continued)

Start Page

SAS Program.sas

+

Run

Cancel

Clear

Copy to flow

Snippet

Log

Results

Output Data (1)

Obs	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	1	0	3	Braund, Mr. Owen Harris	male	22	1	0	A/5 21171	7.2500		S
2	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Thayer)	female	38	1	0	PC 17599	71.2833	C85	C
3	3	1	3	Heikkinen, Miss. Laina	female	26	0	0	STON/O2. 3101282	7.9250		S
4	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35	1	0	113803	53.1000	C123	S
5	5	0	3	Allen, Mr. William Henry	male	35	0	0	373450	8.0500		S

Template-Based Excel Updates with Python and SAS

Template

F2

✕

✓

fx

=A2*0.15*100

	A	B	C	D	E	F
1	Actual Sales	Predicted Sales	Country	Month	TAX	TOTAL
2					0.15	0
3					0.15	0
4					0.15	0
5					0.15	0
6					0.15	0
7					0.15	0
8					0.15	0
9					0.15	0
10					0.15	0
11					0.15	0
12					0.15	0
13					0.15	0
14					0.15	0
15					0.15	0
16					0.15	0
17					0.15	0
18					0.15	0
19					0.15	0
20					0.15	0

<

>

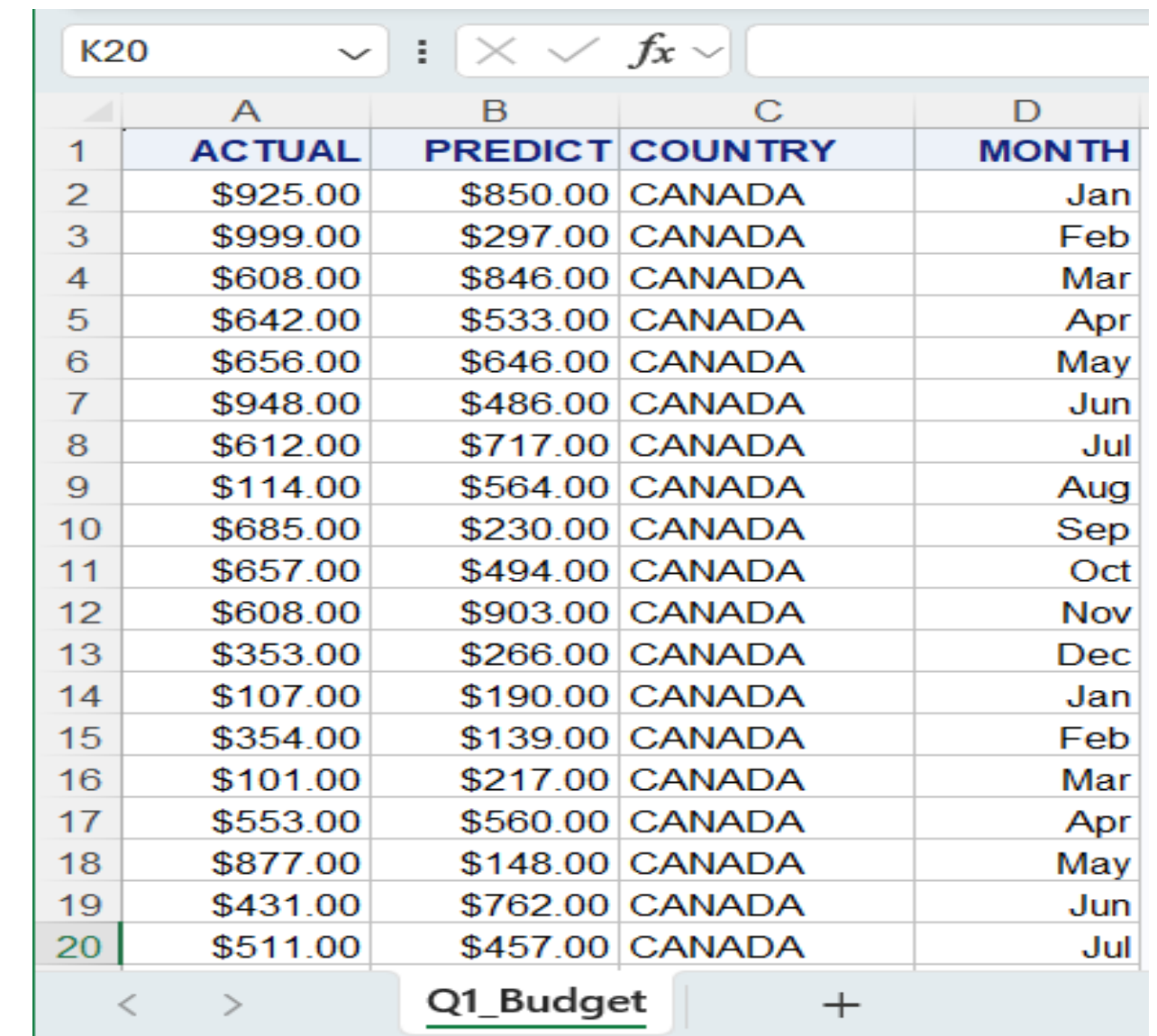
Q1_Budget

+

Template-Based Excel Updates with Python and SAS (Continued)

```
ods excel file="budget.xlsx"  
    options(sheet_name="Q1_Budget");  
  
proc print data=sashelp.prdsale noobs;  
var actual predict country month;  
run;  
  
ods excel close;
```

Source data



	A	B	C	D
1	ACTUAL	PREDICT	COUNTRY	MONTH
2	\$925.00	\$850.00	CANADA	Jan
3	\$999.00	\$297.00	CANADA	Feb
4	\$608.00	\$846.00	CANADA	Mar
5	\$642.00	\$533.00	CANADA	Apr
6	\$656.00	\$646.00	CANADA	May
7	\$948.00	\$486.00	CANADA	Jun
8	\$612.00	\$717.00	CANADA	Jul
9	\$114.00	\$564.00	CANADA	Aug
10	\$685.00	\$230.00	CANADA	Sep
11	\$657.00	\$494.00	CANADA	Oct
12	\$608.00	\$903.00	CANADA	Nov
13	\$353.00	\$266.00	CANADA	Dec
14	\$107.00	\$190.00	CANADA	Jan
15	\$354.00	\$139.00	CANADA	Feb
16	\$101.00	\$217.00	CANADA	Mar
17	\$553.00	\$560.00	CANADA	Apr
18	\$877.00	\$148.00	CANADA	May
19	\$431.00	\$762.00	CANADA	Jun
20	\$511.00	\$457.00	CANADA	Jul

Template-Based Excel Updates with Python and SAS

(continued)

Results

```
proc python infile="/tmp/sample4.py";
run;
```

Q1_Budget						
	A	B	C	D	E	F
1	Actual Sales	Predicted Sales	Country	Month	TAX	TOTAL
2	\$925.00	\$850.00	CANADA	Jan	0.15	13875
3	\$999.00	\$297.00	CANADA	Feb	0.15	14985
4	\$608.00	\$846.00	CANADA	Mar	0.15	9120
5	\$642.00	\$533.00	CANADA	Apr	0.15	9630
6	\$656.00	\$646.00	CANADA	May	0.15	9840
7	\$948.00	\$486.00	CANADA	Jun	0.15	14220
8	\$612.00	\$717.00	CANADA	Jul	0.15	9180
9	\$114.00	\$564.00	CANADA	Aug	0.15	1710
10	\$685.00	\$230.00	CANADA	Sep	0.15	10275
11	\$657.00	\$494.00	CANADA	Oct	0.15	9855
12	\$608.00	\$903.00	CANADA	Nov	0.15	9120
13	\$353.00	\$266.00	CANADA	Dec	0.15	5295
14	\$107.00	\$190.00	CANADA	Jan	0.15	1605
15	\$354.00	\$139.00	CANADA	Feb	0.15	5310
16	\$101.00	\$217.00	CANADA	Mar	0.15	1515
17	\$553.00	\$560.00	CANADA	Apr	0.15	8295
18	\$877.00	\$148.00	CANADA	May	0.15	13155
19	\$431.00	\$762.00	CANADA	Jun	0.15	6465
20	\$511.00	\$457.00	CANADA	Jul	0.15	7665

Template-Based Excel Updates with Python and SAS

(continued)

```
import openpyxl
import shutil

# Load the workbooks
def copy_data_to_template(source_file, template_file, output_file, source_sheet, _template_sheet):
    wb_source = openpyxl.load_workbook(filename=source_file, data_only=True)
    ws_source = wb_source[source_sheet_name]

    # Create a copy of the template file to work with, preserving its original format
    shutil.copyfile(template_file, output_file)
    wb_template = openpyxl.load_workbook(filename=output_file)
    ws_template = wb_template[template_sheet_name]

    # Iterate through source cells and write to template cells
    for row in ws_source.iter_rows():
        for cell in row:
            # Copy the cell value to the corresponding coordinate in the template sheet
            ws_template[cell.coordinate].value = cell.value

    # Save the new file
    wb_template.save(output_file)

# copy_data_to_template('source_data.xlsx', 'template.xlsx', 'filled_template.xlsx', 'Q1_Budget',
# 'Q1_Budget')
```

The FCMP Procedure-Python Objects Overview

- Allows embedded Python code natively inside a SAS session
- Submits and executes functions written in Python from within a SAS session
- Supported using both Viya and SAS 9.4M6+
- The MAS_M2PATH and MAS_PYPATH variables must be configured
- Provides the ability to run SAS and Python at the same time
- Generates a seamless workflow between SAS and Python

The FCMP Procedure-Python Objects Overview

```
proc fcmp;
  /* 1. Declare the Python object */
  declare object py(python);

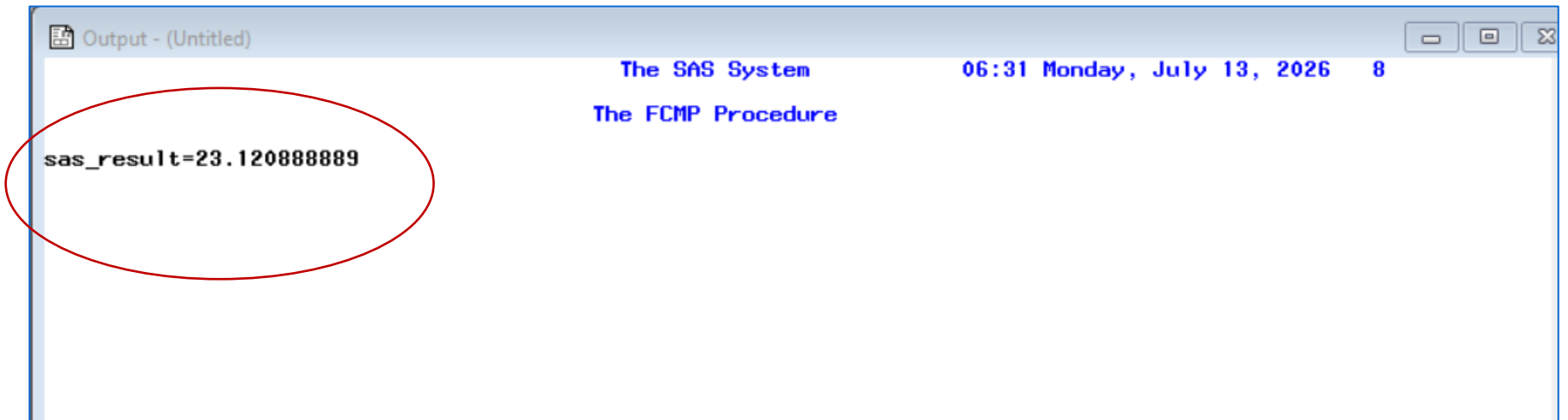
  /* 2. Load the Python code */
  submit into py;
  def BMI_Calculation(val1, val2**2):
    "Output: MyResultKey"
    BMI = 703*(val1 / val2)
    return BMI
  endsubmit;

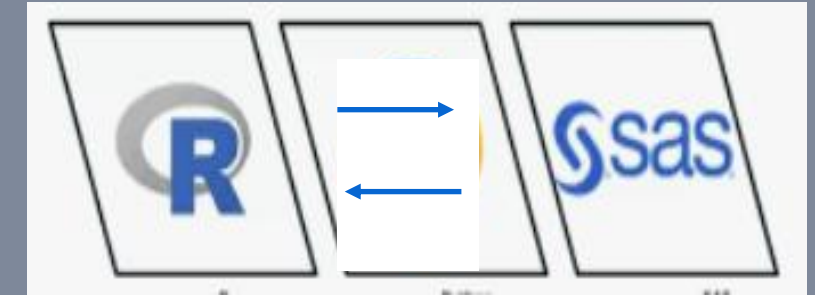
  /* 3. Publish the code to the Python interpreter */
  rc = py.publish();

  /* 4. Execute the Python function */
  rc = py.call("BMI_Calculation", 190, 75);

  /* 5. Retrieve results back into a SAS variable */
  sas_result = py.results["MyResultKey"];
  put sas_result=;

run;
```





Using the SAS Language and R Integration

The R Procedure

- PROC R Fundamentals
- Callback Methods
- Implementation Examples

PROC R Overview

- Released with Viya cadence 2026.03
- Exchange tables and text seamlessly between SAS and R
- Export SAS data sets to R dataFrames via the haven package
- Execute SAS code directly from the R code block
- Integrates the R Program Editor in SAS Studio

The R Procedure

- PROC R Fundamentals
- Callback Methods
- Implementation Examples

PROC R Overview

- Released with Viya cadence 2026.03
- Exchange tables and text seamlessly between SAS and R
- Export SAS data sets to R dataFrames via the haven package
- Execute SAS code directly from the R code block
- Integrates the R Program Editor in SAS Studio

Callback Methods

Method	Results
<code>rplot(r-plot-object, <filename = "plot-name.extension">)</code>	Prints a graph in the Results window
<code>renderImage(full-path-to-image-file)</code>	Renders an image file to the Results window.
<code>show(r-object, <title="title">, <count=number>)</code>	Displays R objects in the Results window
<code>submit("sas-code")</code>	Executes SAS code directly from Python
<code>sasfnc("function-name",args)</code>	Invokes SAS functions
<code>symput("macro-var",value)</code>	Assigns a Python value to SAS macro variable in the global scope
<code>symget("macro-var")</code>	Retrieves the value of a SAS macro variable within the Python Environment
<code>sd2df("sas_dataset")</code>	Exports a SAS data set to a local R data frame
<code>df2sd(df, "sas_dataset")</code>	Transfers an R data frame to a SAS data set

Callback Methods (continued)

Run

Cancel

Clear

Debug

Copy to flow

Snippet

Code

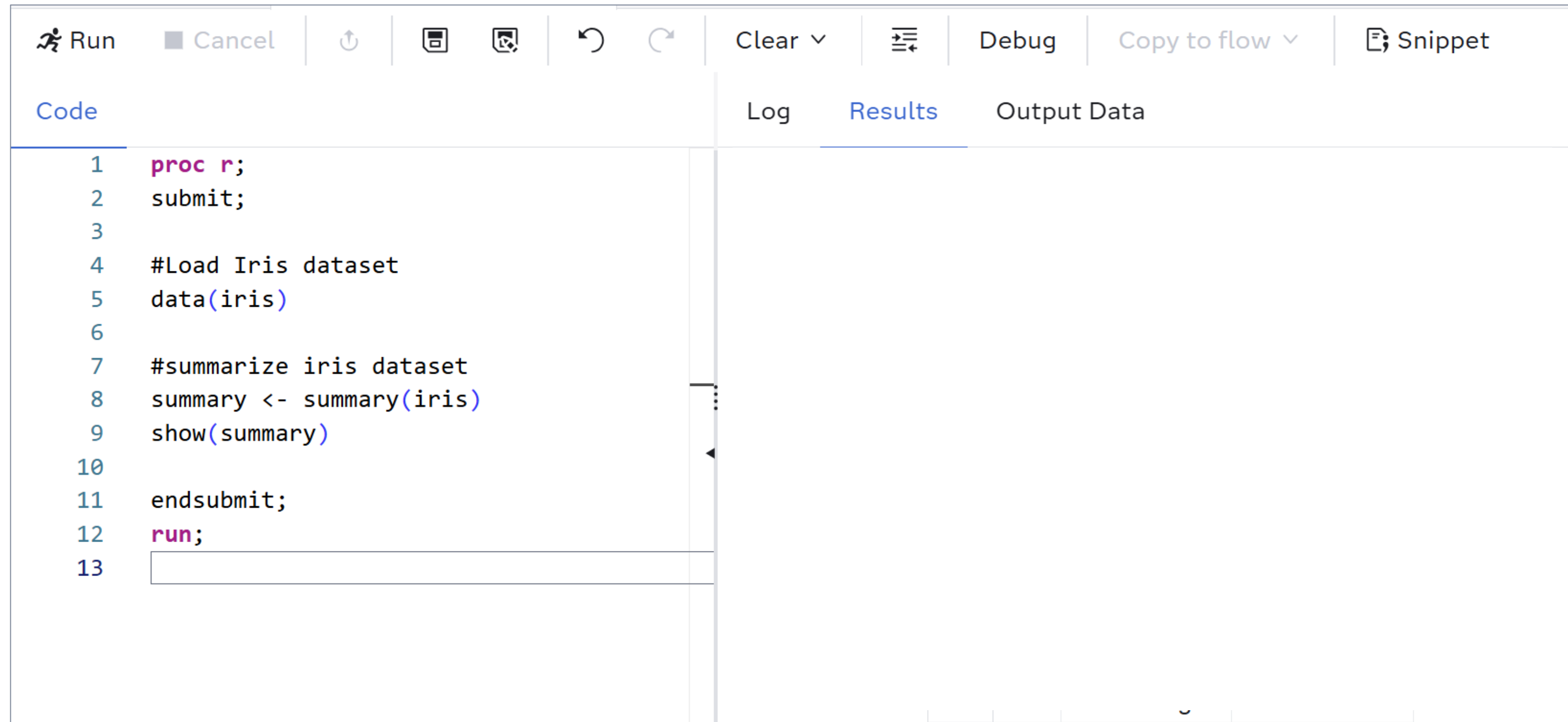
Log

Results

Output Data (1)

```
1  proc r;
2  submit;
3  # Define vectors for each column
4  names <- c("Debra", "John", "Kenneth", "Fred")
5  ages <- c(7, 5, 9, 16)
6  scores <- c(88, 92, 75, 81)
7
8  # Combine into a dataframe
9  df <- data.frame(Name = names, Age = ages, Score = scores)
10
11 # Create macro variable comment
12 symput("comment","Grades from first semester")
13
14 # Transfer dataframe to SAS data set
15 df2sd(df,"work.Grades")
16
17 # Generate report of semester
18 submit("PROC REPORT data=work.Grades;title &comment;run;")
19
20 endsubmit;
21 run;
22
```

PROC R Examples - Displaying R Objects



The screenshot displays the SAS Studio interface. The top toolbar includes buttons for Run, Cancel, and various file and execution actions. Below the toolbar, the 'Code' editor contains the following PROC R code:

```
1  proc r;  
2  submit;  
3  
4  #Load Iris dataset  
5  data(iris)  
6  
7  #summarize iris dataset  
8  summary <- summary(iris)  
9  show(summary)  
10  
11 endsubmit;  
12 run;  
13
```

To the right of the code editor, the 'Results' tab is active, showing a vertical scrollbar. The 'Log' and 'Output Data' tabs are also visible. The bottom of the interface shows a status bar with a small icon.

PROC R Examples – Generating Plots

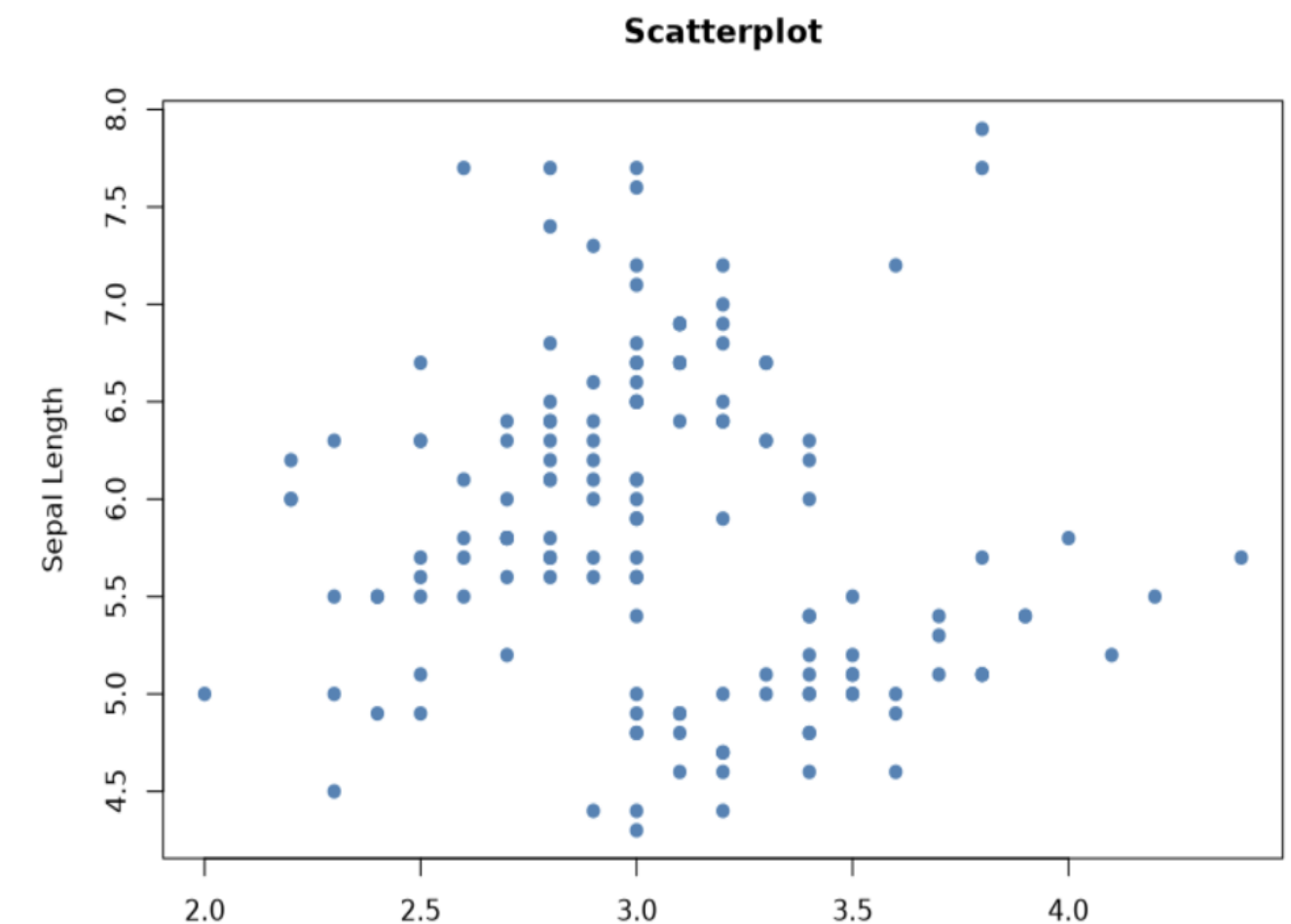
Run Cancel      Clear 

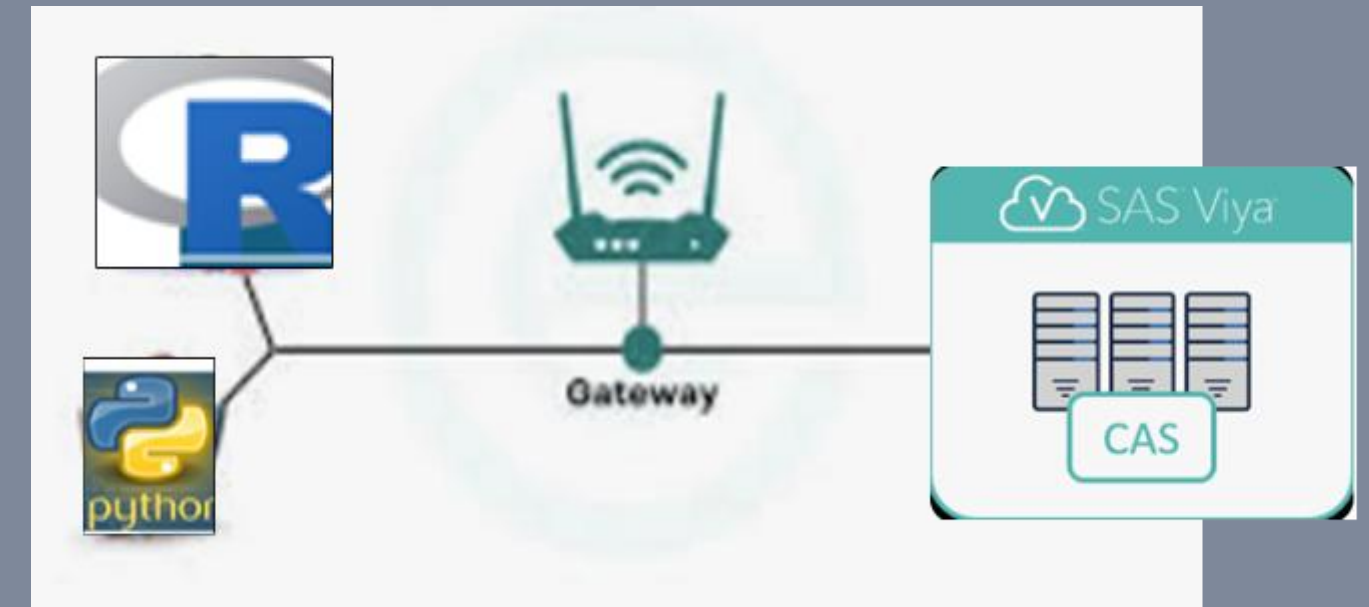
Code

```
1 proc r;  
2 submit;  
3  
4 #Load Iris dataset  
5 data(iris)  
6  
7 # create scatterplot of sepal width vs. sepal length  
8 rplot(quote(plot(iris$Sepal.Width, iris$Sepal.Length,  
9   col='steelblue',  
10  main='Scatterplot',  
11  xlab='Sepal Width',  
12  ylab='Sepal Length',  
13  pch=19))),  
14 filename = "Scaterplot.png")  
15 endsubmit;  
16 run;  
17
```

Clear  Debug Copy to flow  Snippet

Log Results Output Data





The Gateway Procedure

The GATEWAY Procedure Overview

- Became available with Viya Cadence 2024.09
- Enables high efficiency of data transfer between SAS and open source
- Launches multiple Python or R sessions inside the CAS workers
- Requires an active CAS session

The GATEWAY Procedure – Transferring Data

```
proc gateway lang=r;
submit;
  library(arrow)
  # Write a local R data frame to a CAS table
  df <- data.frame(x = 1:10, y = rnorm(10))
  write_table(df, list(caslib= "casuser" name = "myCASdf"))

  # Read a CAS table into an R data frame
  my_data <- read_table(list(caslib="casuser" name = "myCASdf"))
  print(head(my_data))
endsubmit;
run;
```

Managing your Environments in Viya 4

Managing Python and R Environments

- Selecting an IDE to use
- Managing Python and R installed packages
- Determining versions of installed packages

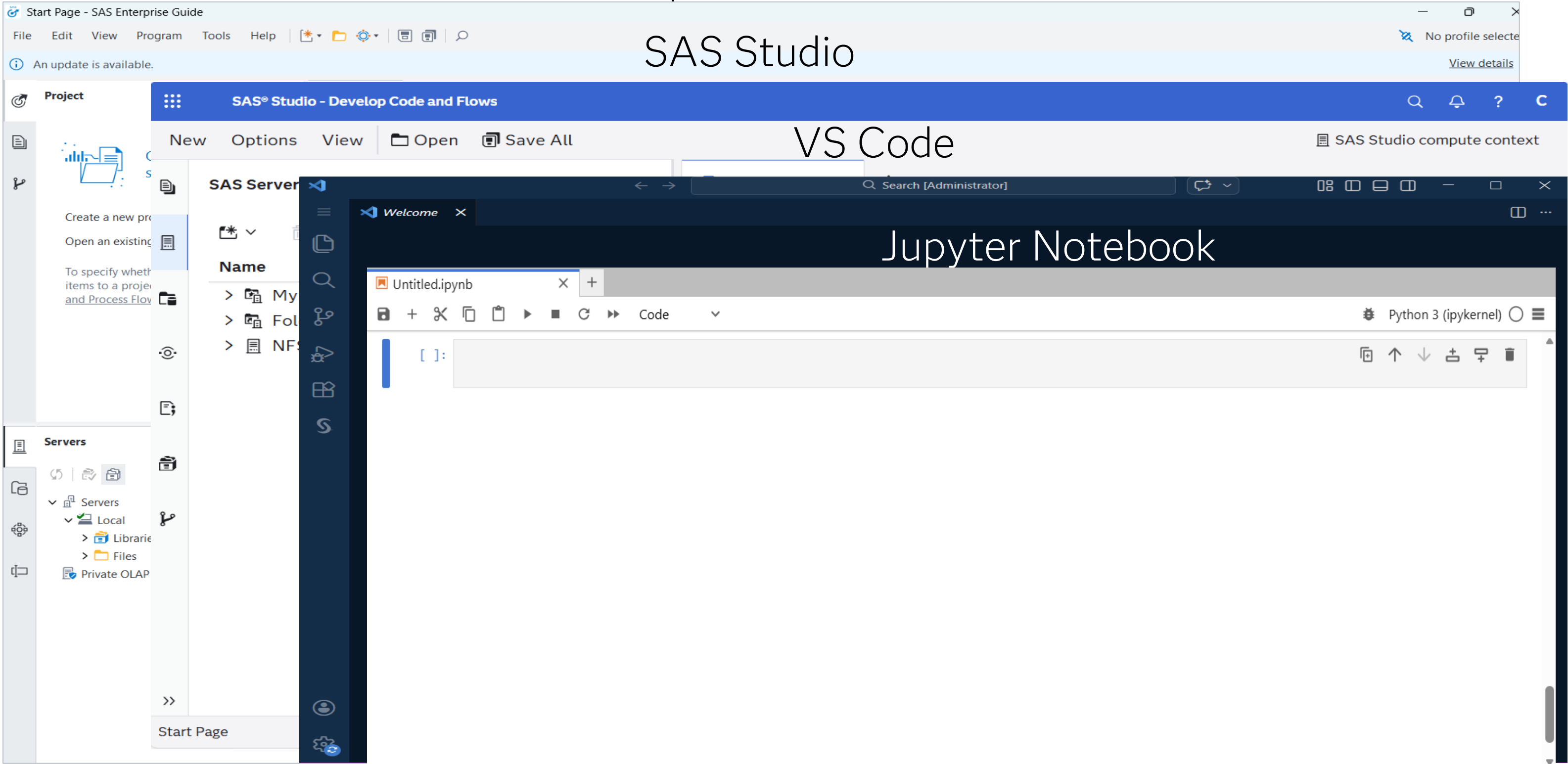
Selecting an IDE to use

SAS Enterprise Guide

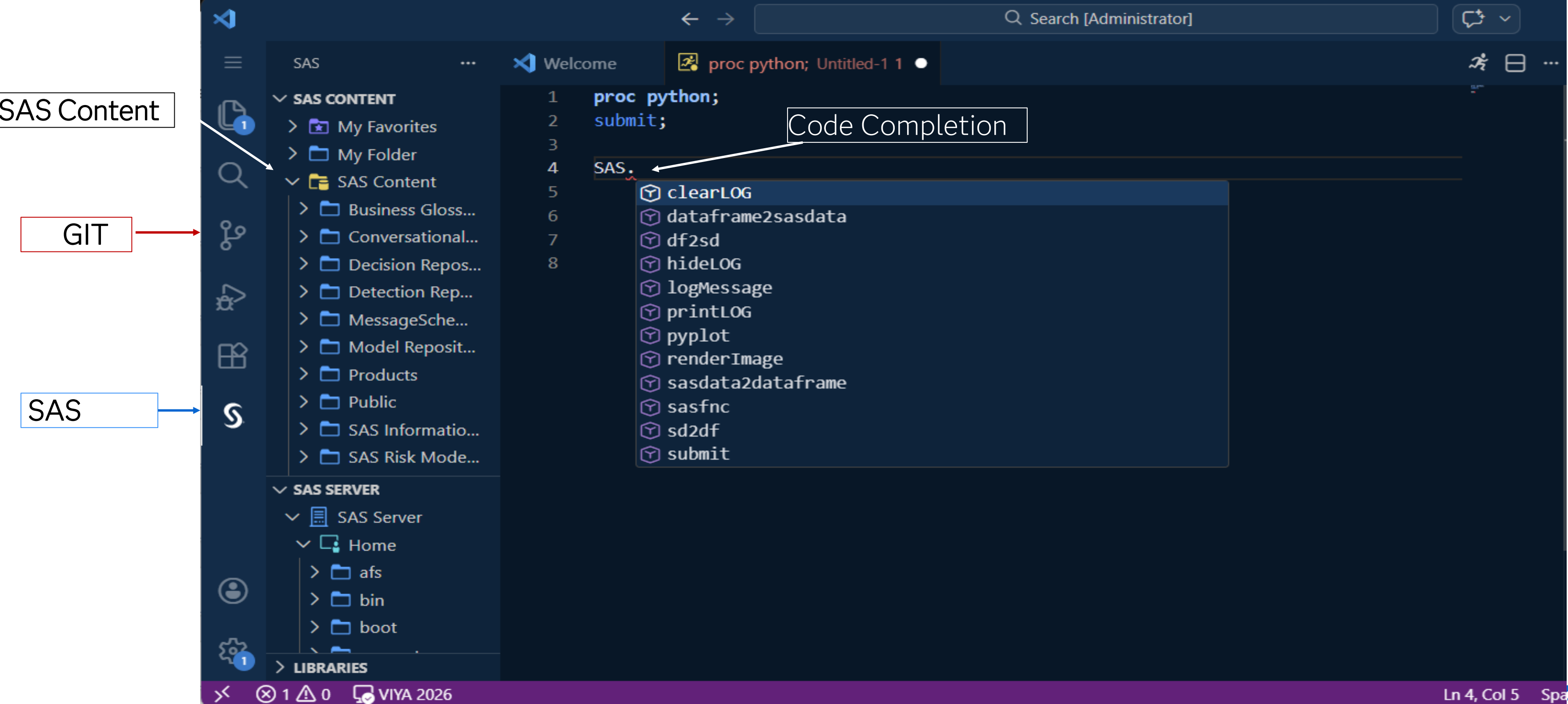
SAS Studio

VS Code

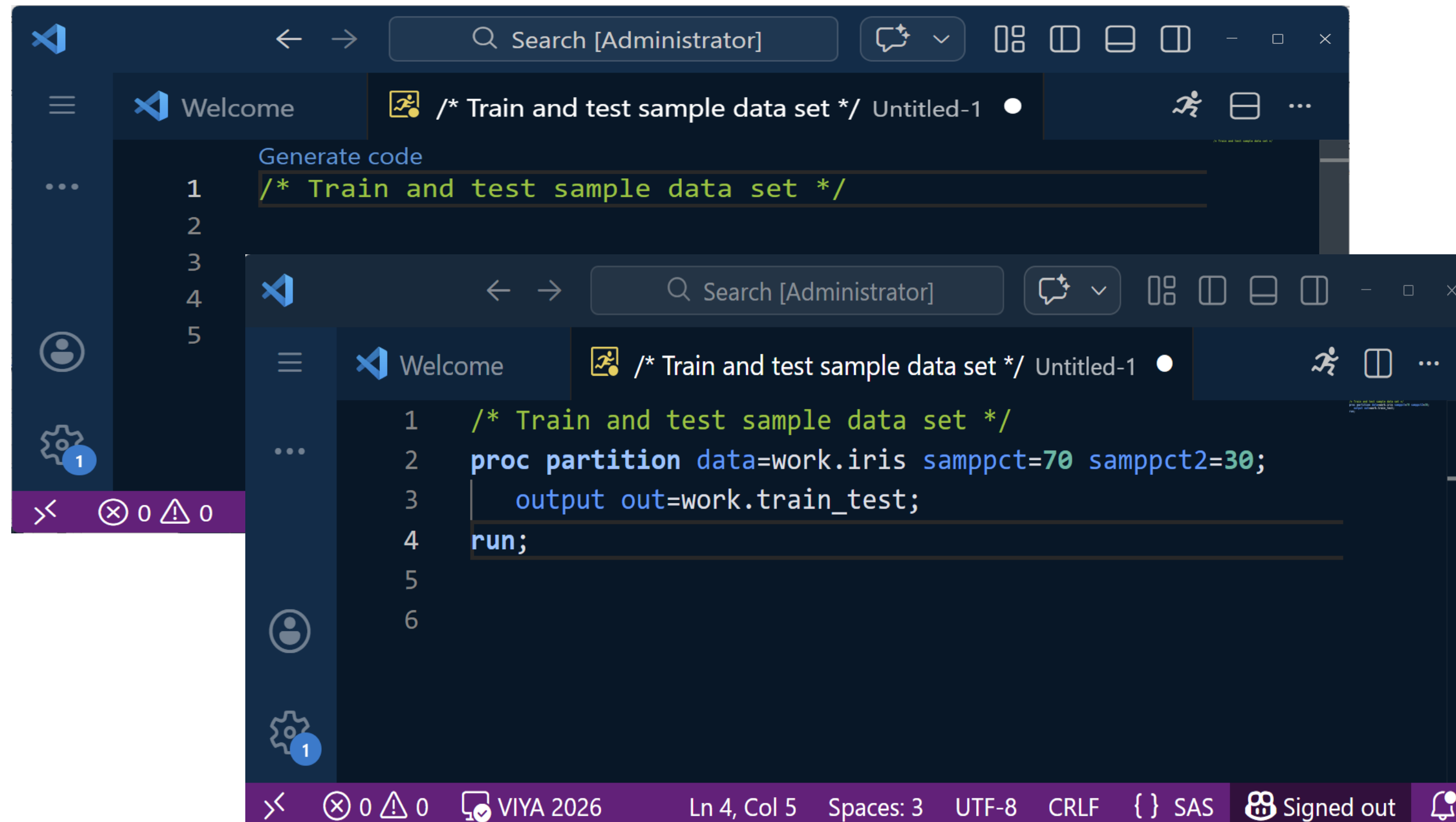
Jupyter Notebook



The SAS Extension for VS Code



The SAS Extension for VS Code – Copilot



Managing Open-Source Packages and Repositories

Managing Open-Source Packages and Repositories

- Installing Python and R for use with Viya 4
- Updating Python and R packages
- Viewing Python and R packages installed

Installing Python and R

- Deploying Python and R using the SAS Configurator for Open-Source
- Making updates to Python and R installations using configuration maps
- Installing Python and R when there is no internet Access

Deploying Python and R using the SAS Configurator for Open-Source

```
change-configur...
51 - op: replace
52   path: /data/default_py.cflags
53   value: "-fPIC"
54 - op: replace
55   path: /data/default_r.cflags
56   value: "-fPIC"
57 - op: replace
58   path: /data/default_py.pip_install_packages
59   value: "Prophet sas_kernel matplotlib sasoptpy sas-espppy NeuralProphet scipy Flask XGBoost TensorFlow pybase64 scikit-learn statsmodels
60   sympy mlxtend Skl2onnx nbeats-pytorch ESRNN onnxruntime opencv-python zipfile38 json2 pyenchant nltk spacy gensim pyarrow great-expectations
61   pandas==2.3.3 numpy==1.26.4"
62 - op: replace
63   path: /data/default_py.pip_r_packages
64   value: "rpy2"
65 - op: replace
66   path: /data/default_py.pip_r_profile
67   value: "default_r"
68 - op: replace
69   path: /data/default_py.python_signer
70   value: https://github.com/Yhg1s.gpg
71 - op: replace
72   path: /data/default_py.python_signature
73   value: https://www.python.org/ftp/python/3.12.12/Python-3.12.12.tgz.asc
74 - op: replace
75   path: /data/default_py.python_tarball
76   value: https://www.python.org/ftp/python/3.12.12/Python-3.12.12.tgz
77 - op: replace
78   path: /data/default_r.r_tarball
79   value: https://cloud.r-project.org/src/base/R-4/R-4.4.3.tar.gz
80 - op: replace
81   path: /data/default_r.packages
82   value: "R6 dplyr jsonlite httr tidyverse randomForest xgboost forecast arrow logger knitr haven svglite plotly"
83 - op: replace
84   path: /data/default_r.pkg_repos
85   value: "https://cran.rstudio.com/ http://cran.rstudio.com/ https://cloud.r-project.org/ http://cloud.r-project.org/"
86 target:
87   version: v1
88   kind: ConfigMap
89   name: sas-pyconfig-parameters
```

Python Packages

Python Download

R Download

R Packages

Installing Open-Source Python Packages within your local Site-Info

Run

Code

```
1 proc python;
2 submit;
3 import subprocess
4 import sys
5 subprocess.run([sys.executable, "-m", "pip", "install", "beautifulsoup4"], check=True)
6 endsubmit;
7 run;
```

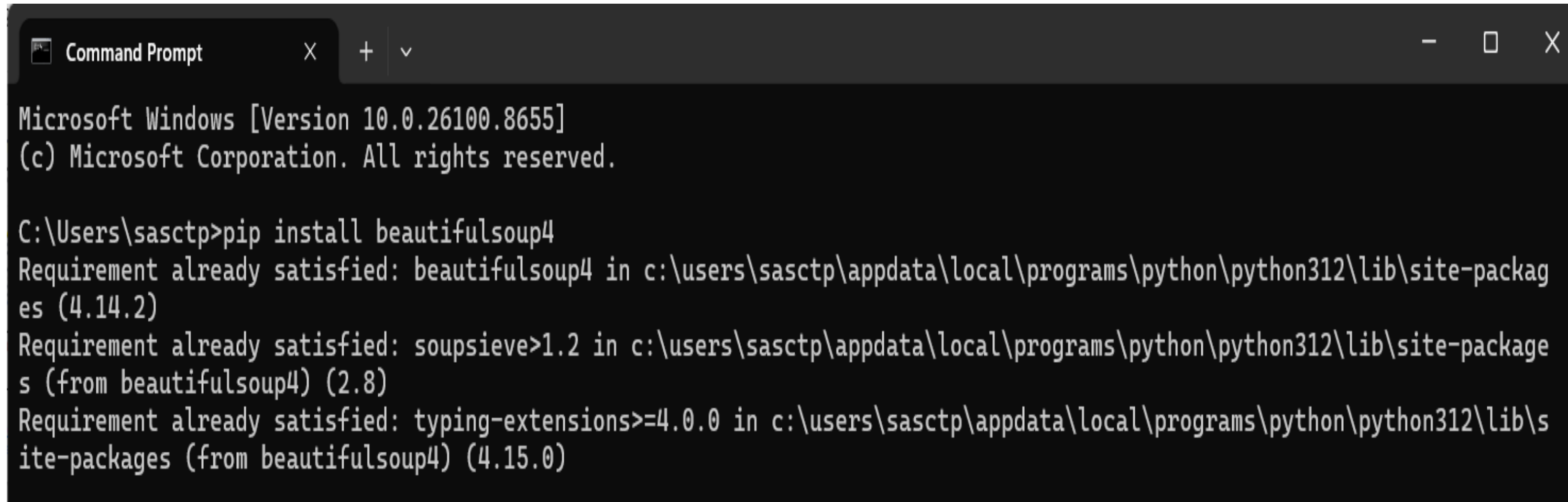
LogResultsOutput Data

⊗ (0)⚠ (0)ℹ (10)

87 endsubmit;
88 run;
>>>
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: beautifulsoup4 in /opt/sas-pyconfig/default_py/lib/python3.11/site-packages (4.14.3)
Requirement already satisfied: soupsieve>=1.6.1 in /opt/sas-pyconfig/default_py/lib/python3.11/site-packages (from
(2.8)
Requirement already satisfied: typing-extensions>=4.0.0 in /opt/sas-pyconfig/default_py/lib/python3.11/site-package
beautifulsoup4) (4.15.0)
[notice] A new release of pip is available: 25.3 -> 26.0.1
[notice] To update, run: /opt/sas-pyconfig/default_py/bin/python3 -m pip install --upgrade pip
NOTE: PROCEDURE PYTHON used (Total process time):
real time6.98 seconds
cpu time0.13 seconds

sas-studio-custom-steps/Python - Create a Virtual Environment at main ·
sassoftware/sas-studio-custom-steps

Installing Open-Source Python Packages within your local Site-Info



```
Microsoft Windows [Version 10.0.26100.8655]
(c) Microsoft Corporation. All rights reserved.

C:\Users\sasctp>pip install beautifulsoup4
Requirement already satisfied: beautifulsoup4 in c:\users\sasctp\appdata\local\programs\python\python312\lib\site-packages (4.14.2)
Requirement already satisfied: soupsieve>1.2 in c:\users\sasctp\appdata\local\programs\python\python312\lib\site-packages (from beautifulsoup4) (2.8)
Requirement already satisfied: typing-extensions>=4.0.0 in c:\users\sasctp\appdata\local\programs\python\python312\lib\site-packages (from beautifulsoup4) (4.15.0)
```

Viewing installed Python Packages

```
proc python;  
submit;  
import subprocess  
import sys  
subprocess.run([sys.executable, "-m", "pip", "list"])  
endsubmit;  
run;
```

```
>>>  
Package                                Version    Build  
-----  
absl-py                                2.3.1  
aiohappyeyeballs                        2.6.1  
aiohttp                                3.13.2  
aioice                                  0.10.2  
aiortc                                  1.14.0  
aiosignal                              1.4.0  
altair                                  4.2.2  
annotated-types                         0.7.0
```

Viewing Installed R Packages

```
proc R;  
submit;  
packinfo <- installed.packages(fields = c("Package", "Version"))  
packinfo[,c("Package", "Version")]  
endsubmit;  
run;
```

	Package	Version
arrow	"arrow"	"23.0.1.2"
askpass	"askpass"	"1.2.1"
assertthat	"assertthat"	"0.2.1"
backports	"backports"	"1.5.1"
base	"base"	"4.3.3"
base64enc	"base64enc"	"0.1-6"
bit	"bit"	"4.6.0"
bit64	"bit64"	"4.6.0-1"
blob	"blob"	"1.3.0"
boot	"boot"	"1.3-29"
broom	"broom"	"1.0.12"
bslib	"bslib"	"0.10.0"
cachem	"cachem"	"1.1.0"
callr	"callr"	"3.7.6"

Common Python and R Integration Errors

Error: *Unable to create Python subprocess*

Error: *The PROC PYTHON procedure timed out after*

Error: *Cannot find python3*

Error: *Unable to create Python subprocess- A memory error may have occurred*

Error: *The PROC Python procedure is not found*

Running your Analysis on the Cloud and Compute

Running your Analysis in the Cloud

- Highlights of the CAS server
- Connecting to the CAS server
- Running your analysis on the CAS server

Highlights of the CAS server

- *In-Memory Processing*
- *Scalability*
- *Parallel processing*
- *Integration*
- *Flexibility*

Connecting to the CAS server using Python

Using an HTTP Ingress connection on the local machine

```
import swat
import os
os.environ['CAS_CLIENT_SSL_CA_LIST'] = "c:\temp\trustedcerts.pem"
conn=swat.CAS('https://my-cas-host.com:443/cas-shared-default-http/,
username,password)
```

Using a BINARY (CAS) connection on the local machine

```
import swat
import os
os.environ['CAS_CLIENT_SSL_CA_LIST']="c:\temp\trustedcerts.pem"
conn=swat.CAS('host',port=5570,username,password)
```

Connecting to the CAS server using Python (continued)

https://viya4msft.ondemand.sas.com/SASLogon/oauth/authorize?client_id=SWAT&response_type=code



Connecting to the CAS server using Python

```
import swat
import os
os.environ['CAS_CLIENT_SSL_CA_LIST'] = "c:\temp\trustedcerts.pem"
conn=swat.CAS('https://my-cas-host.com:443/cas-shared-default-
http/,authcode='authode')
```

Running your Analysis in the Cloud

Log

Results

Output Data

⊗ (0)

⚠ (0)

ℹ (10)

...

•>>

NOTE: Cloud Analytic Services made the uploaded file available as table MY_SAS_TABLE5 in caslib CASUSER(sasctp).

NOTE: The table MY_SAS_TABLE5 has been created in caslib CASUSER(sasctp) from binary data uploaded to Cloud Analytic Services.

[Fetch]

Selected Rows from Table MY_SAS_TABLE5

	OBJECTID	HOOD	FClosure	Poverty	Seniors	Renters
0	1.0	Central Oakland	1.0	64.8%	3.82%	86.2%
1	2.0	North Oakland	8.0	39.2%	6.72%	70.7%
2	3.0	West Oakland	1.0	46.6%	8.06%	61.6%
3	4.0	Crawford-Roberts	2.0	48.2%	16.02%	75.5%
4	5.0	Strip District	20.0	2.1%	17.27%	63.0%
5	83.0	Middle Hill	5.0	26.4%	22.34%	55.7%
6	6.0	Central Business District	6.0	28.8%	14.46%	76.0%
7	7.0	Spring Hill-City View	7.0	33.7%	11.63%	50.3%
8	8.0	Perry South	15.0	37.0%	10.61%	53.1%
9	9.0	Summer Hill	0.0	7.9%	9.89%	35.3%

Elapsed: 0.00165s, user: 0.00136s, sys: 0.000132s, mem: 0.931mb

•>>

NOTE: PROCEDURE PYTHON used (Total process time):

real time

3.61 seconds

cpu time

0.06 seconds

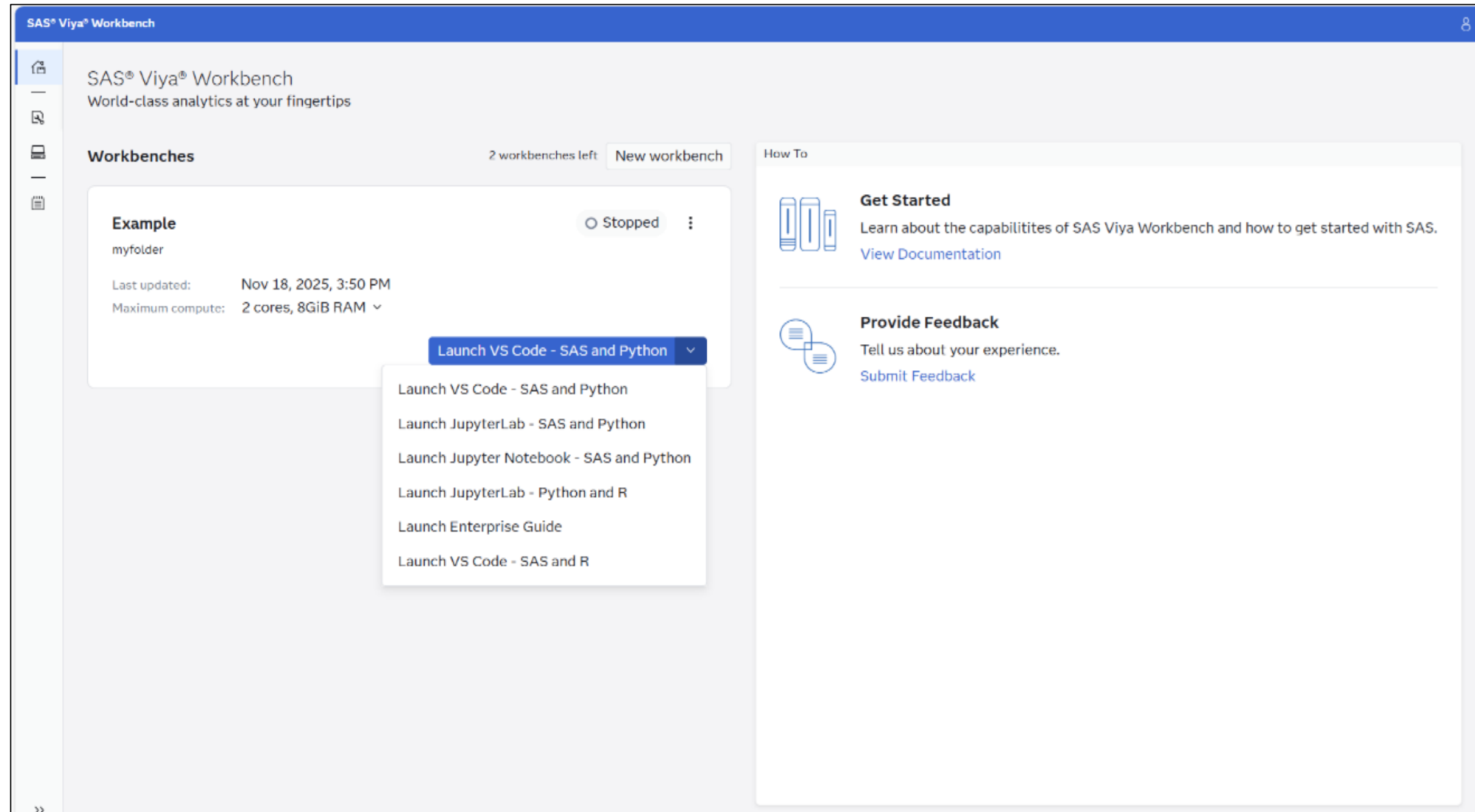


Introduction to SAS Viya Workbench

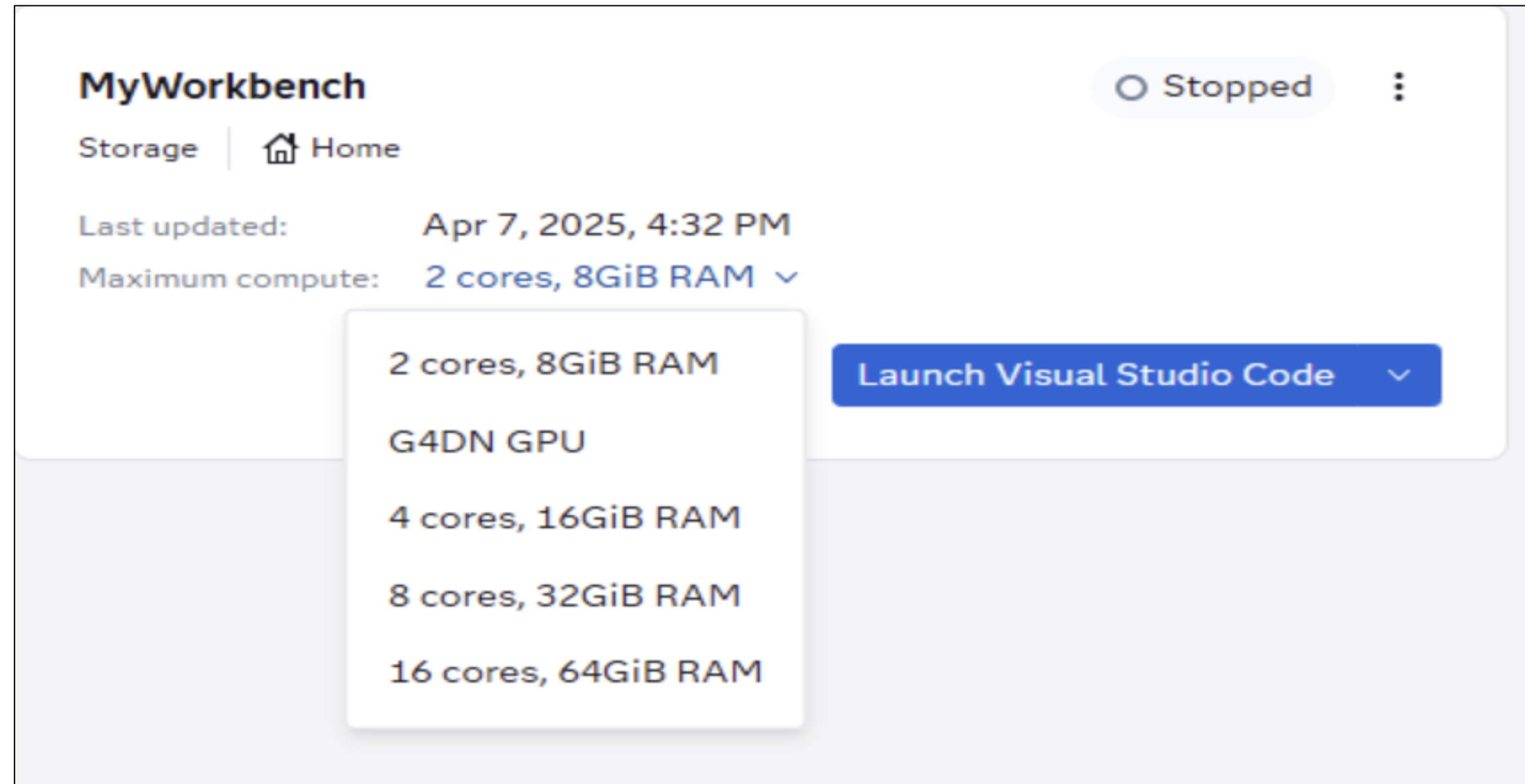
SAS Viya Workbench Overview

- Work in preferred coding languages (SAS, Python, or R)
- Provides the ability to run SAS 9 code and SAS Viya analytics
- Scalable Compute Power such as CPU/GPU based on workload requirements
- Minimize cloud computing cost with self-terminating sessions
- Seamless Integration & Collaboration with GIT integration and import of projects
- Can install the packages needed in this environment without IT support

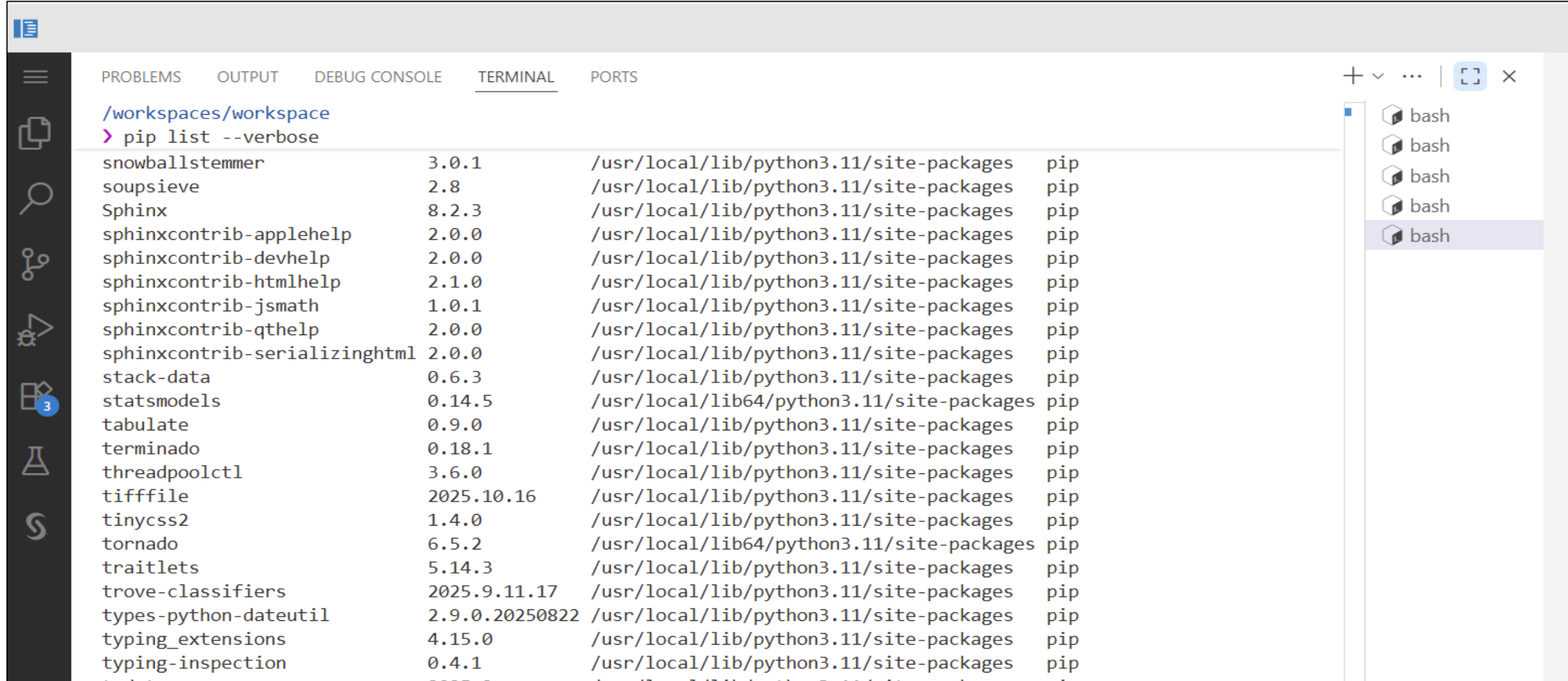
SAS Viya Workbench – Selecting an IDE



SAS Viya Workbench - Scalable



SAS Viya Workbench – Installing Packages



The screenshot shows the SAS Viya Workbench interface with a terminal window open. The terminal displays the output of the command `pip list --verbose`, listing various installed Python packages. The interface includes a sidebar with icons for file explorer, search, and other tools. The terminal window has tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. The current terminal session is titled `/workspaces/workspace`.

Package Name	Version	Location	Installer
snowballstemmer	3.0.1	/usr/local/lib/python3.11/site-packages	pip
soupsieve	2.8	/usr/local/lib/python3.11/site-packages	pip
Sphinx	8.2.3	/usr/local/lib/python3.11/site-packages	pip
sphinxcontrib-applehelp	2.0.0	/usr/local/lib/python3.11/site-packages	pip
sphinxcontrib-devhelp	2.0.0	/usr/local/lib/python3.11/site-packages	pip
sphinxcontrib-htmlhelp	2.1.0	/usr/local/lib/python3.11/site-packages	pip
sphinxcontrib-jsmath	1.0.1	/usr/local/lib/python3.11/site-packages	pip
sphinxcontrib-qthelp	2.0.0	/usr/local/lib/python3.11/site-packages	pip
sphinxcontrib-serializinghtml	2.0.0	/usr/local/lib/python3.11/site-packages	pip
stack-data	0.6.3	/usr/local/lib/python3.11/site-packages	pip
statsmodels	0.14.5	/usr/local/lib64/python3.11/site-packages	pip
tabulate	0.9.0	/usr/local/lib/python3.11/site-packages	pip
terminado	0.18.1	/usr/local/lib/python3.11/site-packages	pip
threadpoolctl	3.6.0	/usr/local/lib/python3.11/site-packages	pip
tifffile	2025.10.16	/usr/local/lib/python3.11/site-packages	pip
tinycss2	1.4.0	/usr/local/lib/python3.11/site-packages	pip
tornado	6.5.2	/usr/local/lib64/python3.11/site-packages	pip
traitlets	5.14.3	/usr/local/lib/python3.11/site-packages	pip
trove-classifiers	2025.9.11.17	/usr/local/lib/python3.11/site-packages	pip
types-python-dateutil	2.9.0.20250822	/usr/local/lib/python3.11/site-packages	pip
typing_extensions	4.15.0	/usr/local/lib/python3.11/site-packages	pip
typing-inspection	0.4.1	/usr/local/lib/python3.11/site-packages	pip

Contact

Name: Chevell Parker

Company: SAS Institute Inc.

City/State: Cary, NC

Email: Chevell.parker@sas.com



Resources

Samples

https://github.com/Chevell1/Innovate_26.git



Q&A

